Instructions for Lab Submission

1. **File Naming and Submission:**
   a. Submit your solution as a **ZIP file** named `lab<number>_<ERP>.zip`, where `<ERP>` is your ERP number. For example, if your ERP number is 12345, the file should be named `lab1_12345.zip`.
   b. Ensure that the ZIP file contains:
      i. C++ source files named as `task<number>.cpp` (e.g., `task1.cpp`).
      ii. No unnecessary files (such as executables, temporary files, or editor configs).
   c. Do not submit **TEXT(.txt)** or **EXE(.exe)** files.
2. **Code Organization:**
   a. Write each task in its own `.cpp` file (as specified above).
   b. Ensure that your code is modular and logically separated into functions when specified.
3. **Code Neatness:**
   a. Ensure your code is **well-formatted** and easy to read. Use proper **indentation** and **meaningful variable names**.
   b. Include appropriate **comments** to explain the logic where necessary (especially for functions and complex sections of the code).
   c. **Add clear explanations where required (or specified)** to make the logic of your program easy to follow.
4. **Code Compilation and Functionality:**
   a. Ensure your code compiles without errors or warnings. You should be able to compile and run your code on any system with a standard C++ compiler (e.g., GCC, Clang, MSVC).
   b. Double-check that the functionality of the program meets the requirements outlined in the task.
5. **Additional Instructions:**
   a. Do not use any libraries or features that have not been covered in the course material, unless specifically instructed.
   b. Make sure your code adheres to the principles of good software development (e.g., modular functions, minimal repetition, and no hardcoding).
   c. If you encounter any issues or require clarifications, reach out before the submission deadline.
6. **Deadline:**
   a. Ensure that you submit your lab on time, before the deadline. Late submissions will not be accepted
   b. Double-check that the zip file is correctly named and contains all necessary files before submitting.
7. **Plagiarism Policy:**
   a. The work submitted must be entirely your own. Any code or content copied from others (including online sources, classmates or AI) will result in an automatic zero for the task.

Lecture Review

Setting up g++ Compiler

Check whether g++ is installed:

```
g++ --version
```

If it is not recognized, install MSYS2 and add the g++ compiler to PATH.

For Windows, follow: `https://www.msys2.org/`

In the lab, g++ is already installed.

Compiling and Running a C++ Program

Save your program with a `.cpp` extension. In the terminal:

```
g++ program.cpp -o program.exe
./program.exe
```

Variables: Declaration and Assignment

A variable stores a value and must have a data type.

```
int age;
double price;
```

Values can be assigned using `=`:

```
age = 20;
price = 15.5;
```

Declaration and assignment can also be combined:

```
int age = 20;
```

Input and Output

Use `cin` to take input and `cout` to display output.

```
int age;
cin >> age;
cout << "Age: " << age;
```

Ternary Operator

The ternary operator is a short form of `if-else`.

```
condition ? value1 : value2;
```

Example:

```
int age = 20;
cout << (age >= 18 ? "Adult" : "Minor");
```

Ternary operators can also be nested.

```
cout << (marks >= 80 ? "A" :
marks >= 70 ? "B" : "C");
```

if-else

Use `if-else` to choose between two possible cases.

```
if (age >= 18) {
cout << "Adult";
} else {
cout << "Minor";
}
```

Nested if-else

An `if-else` statement can be placed inside another `if` or `else`.

```
if (age >= 18) {
    if (age >= 60)
        cout << "Senior";
    else
        cout << "Adult";
    }
else {
    cout << "Minor";
}
```

Mathematical Operators

Common mathematical operators:

- Addition: `+`
- Subtraction: `-`
- Multiplication: `*`
- Division: `/`
- Remainder: `%`

Example:

```
int result = 10 + 5 * 2;
```

Logical Operators

Logical operators are used to combine conditions.

- AND: `&&`
- OR: `||`
- NOT: `!`

Examples:

```
age >= 18 && age <= 30
age < 18 || age > 60
!(age == 20)
```

Type Casting

Type casting is used to convert a value from one data type to another.

For example, converting an integer to a `double`:

```
int total = 85;
double average = (double) total / 3;
```

Explicit type casting is useful when integer division would otherwise produce an integer result.

Lab Tasks

Task 1: Number Guessing Challenge

Write a C++ program that asks the user to enter a secret number within a predefined range and then allows the user to guess the number.

Problem Description

The program should use a fixed range of **1 to 100**. First, ask the user to enter a number within this range. Validate the entered number using `if-else`. If the number is outside the range, display an appropriate message and end the program.

If the number is valid, ask the user to enter a guess. Compare the guess with the first number and display an appropriate message indicating whether the guess is:

- Lower than the number
- Greater than the number
- Exactly equal to the number

Requirements

- Take two integer inputs from the user.
- Use `if-else` to validate the first number.
- Use `if-else` to compare the guess with the number.
- Display an appropriate message for each case.
- The valid range must be fixed in the program as 1 to 100.

Test Case

```
Input:
Enter a number (1-100): 65
Enter your guess: 42

Output:
Your guess is lower than the number.
```

Output

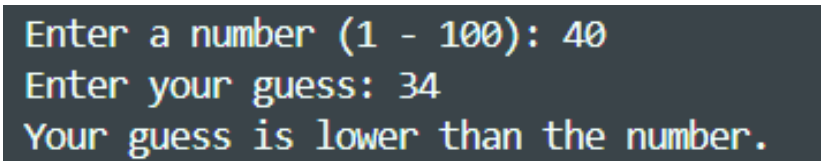

Figure 1: Task 1 Output

Task 2: Leap Year Checker

Write a C++ program that asks the user to enter a year and determines whether the year is a leap year.

Problem Description

A year is considered a leap year if:

- It is divisible by 4, and
- It is not divisible by 100, unless
- It is also divisible by 400.

Use **nested** `if-else` **statements** to determine and display whether the entered year is a leap year or not.

Optional: You may use **nested ternary conditional operators** as an alternative approach.

Requirements

- Take the year as an integer input.
- Use nested `if-else` statements.
- Apply the given leap year rules.
- Display either **Leap Year"** or **Not a Leap Year"**.
- **Optional:** Use the ternary conditional operator `?:`.
- **Optional:** Use nested ternary conditional operators.

Test Case

```
Input:
Enter year: 2024

Output:
2024 is a Leap Year.
```

Output

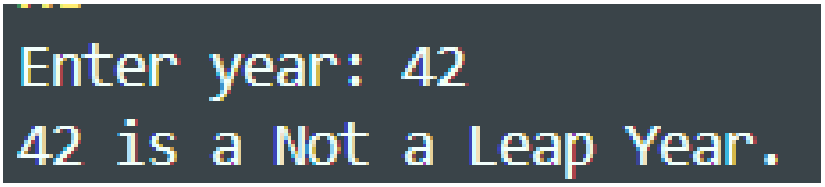

Figure 2: Task 2 Output

Task 3: Student Eligibility Checker

Write a C++ program that checks whether a student is eligible for a particular activity based on their marks and attendance.

Problem Description

The program should take the student's marks and attendance percentage as input.

A student is eligible if:

- Their marks are at least 50 and their attendance is at least 75%, or
- Their marks are at least 80 regardless of attendance.

Use mathematical and logical operators to perform the required checks. Display an appropriate message indicating whether the student is **Eligible** or **Not Eligible**.

Requirements

- Take marks and attendance as input from the user.
- Use comparison operators.
- Use AND and OR logical operators.
- Use `if-else` to determine the result.
- Display the student's eligibility status.

Test Case

```
Input:
Enter marks: 72
Enter attendance: 80

Output:
Student is Eligible.
```

Output

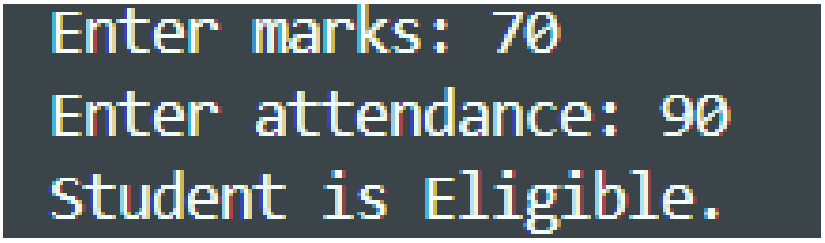

Figure 3: Task 3 Output

Task 4: Number Forensics

Write a C++ program that performs a mathematical analysis of a three-digit number entered by the user.

Problem Description

The program should ask the user to enter a three-digit positive integer. Extract its hundreds, tens, and units digits using arithmetic operations and integer division.

The program should then calculate the sum and product of its digits and create a new number by swapping the first and last digits.

The program must also determine whether the digit sum is even or odd and whether the original number is divisible by 3. Finally, display all information in a formatted **Number Forensics Report**.

Requirements

- Take one three-digit integer as input.
- Extract all three digits using arithmetic operations and integer division.
- Calculate the sum and product of the three digits.
- Create the swapped number by exchanging the first and last digits.
- Use `if-else` to determine whether the digit sum is even or odd.
- Use `if-else` to determine whether the number is divisible by 3.
- Use comparison and logical operators where appropriate.

- **Optional:** Use `setw()` to align the report.
- **Optional:** Use string concatenation to create the report heading.
- **Optional:** Do not use the modulus operator `%`.
- Do not use loops, arrays, functions, or additional libraries.

Output

```
Enter a three-digit number: 368

========================================
        NUMBER FORENSICS REPORT
========================================
Original Number:      368
Hundreds Digit:       3
Tens Digit:           6
Units Digit:          8
Digit Sum:            17
Digit Product:        144
Swapped Number:       863
Digit Sum:            Odd
Divisible by 3:       No
========================================
```

Figure 4: Task 4 Output

Task 5: Smart Access Verification

Write a C++ program that simulates a university smart access system. The system must determine whether a student can enter a restricted laboratory.

Problem Description

The program should ask the user for the student's name, age, ID's last digit, and number of previous failed attempts.

Access is granted only when all of the following conditions are satisfied:

- The student is at least 18 years old.
- The ID's last digit is even.
- The number of failed attempts is less than 3.

The program should first verify the age. If the student is eligible by age, use nested `if-else` statements to perform the remaining security checks.

If access is granted, calculate a security score using:

$$\text{Score} = \text{Age} + (\text{ID Digit} \times 10) - (\text{Failed Attempts} \times 5)$$

Also calculate the score as a percentage of a maximum score of 150 using explicit type casting.

Requirements

- Take the required information as input.
- Use comparison and logical operators to test the conditions.
- Use nested `if-else` statements.
- Use at least one condition combining multiple operators.
- Determine whether the ID's last digit is even.
- Calculate the security score using arithmetic operators.
- Use explicit type casting when calculating the percentage.
- **Optional:** Use `setw()` to align the final report.
- **Optional:** Use string concatenation in the report heading.
- Do not use loops, arrays, functions, or the ternary operator.

Output

```
Enter student name: Ali
Enter age: 18
Enter last digit of student ID: 5
Enter failed attempts: 7

========================================
            Ali - ACCESS REPORT
========================================
Age:                18
ID Last Digit:      5
Failed Attempts:    7
Access Status:      DENIED
Reason:             Security requirements failed
========================================
```

Figure 5: Task 5 Output

Task 6: Three-Number Transformation

Write a C++ program that transforms three integers using arithmetic and assignment operators without using a temporary variable.

Problem Description

The program should ask the user to enter three integers `A`, `B`, and `C`. Perform the following operations in exactly this order:

1. Swap `A` and `B`.

2. Swap `B` and `C`.
3. Swap `A` and `C`.

After completing the transformations, calculate the sum and average of the final values. Also determine the largest and smallest final values and their difference.

Requirements

- Take three integer inputs named `A`, `B`, and `C`.
- Perform all three swaps in the specified order.
- Do not use a temporary variable for swapping.
- Use arithmetic and assignment operators to perform the swaps.
- Calculate the sum of the final values.
- Use explicit type casting to calculate the average correctly.
- Use `if-else` to determine the largest and smallest values.
- Calculate the difference between the largest and smallest values.
- **Optional:** Use `setw()` to align the output.
- **Optional:** Use string concatenation to create the report heading.
- Do not use arrays, loops, functions, or `sort()`.

Output

```
Enter A: 78
Enter B: 90
Enter C: 34

========================================
          NUMBER TRANSFORMATION
========================================
Final A:              78
Final B:              34
Final C:              90
Sum:                  202
Average:              67.33
Largest:              90
Smallest:             34
Difference:           56
========================================
```

Figure 6: Task 6 Output